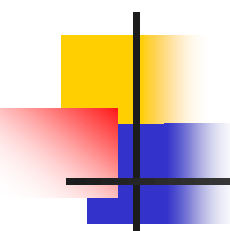


TRÍ TUỆ NHÂN TẠO

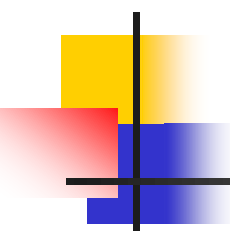
Giải quyết vấn đề bằng tìm kiếm trên không gian trạng thái

GS. TS. Nguyễn Thanh Thuỷ
Khoa Công nghệ Thông tin
Đại học Bách khoa Hà nội



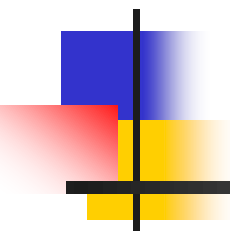
Nội dung

- Biểu diễn vấn đề bằng không gian trạng thái
- Giải quyết vấn đề bằng cách tìm kiếm trên không gian trạng thái
- Các phương pháp tìm kiếm không sử dụng tri thức
- Các phương pháp tìm kiếm sử dụng tri thức
- Tìm kiếm trên cây VÀ/HOẶC

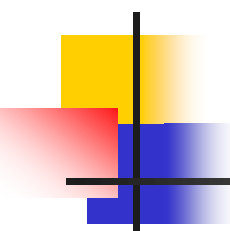


Giải quyết vấn đề

- Mục đích : Tìm một chuỗi các hành động để đạt đến đích của vấn đề
- Hai phương án giải quyết vấn đề
 - Giải quyết lập tức (offline)
 - Lập kế hoạch giải quyết khi đã có hiểu biết toàn bộ vấn đề đặt ra
 - Giải thuật cho kết quả tức thời
 - Không có yếu tố bất ngờ
 - Giải quyết từng bước (online):
 - Tại thời điểm xuất phát, chỉ hiểu biết một phần về vấn đề
 - Ra quyết định ở từng thời điểm cho tới khi đạt tới kết quả
 - Có yếu tố bất ngờ

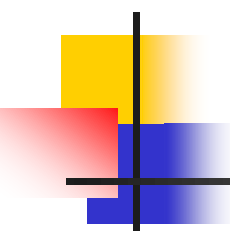


Biểu diễn vấn đề bằng không gian trạng thái



Không gian trạng thái

- Biểu diễn vấn đề
 - Trạng thái xuất phát
 - Chuyển trạng thái (hành động)
 - Trạng thái đích
 - Một hoặc nhiều trạng thái được quyết định bởi một phép thử nào đó
- Giải pháp : Chuỗi các phép chuyển trạng thái để chuyển từ trạng thái xuất phát về trạng thái đích

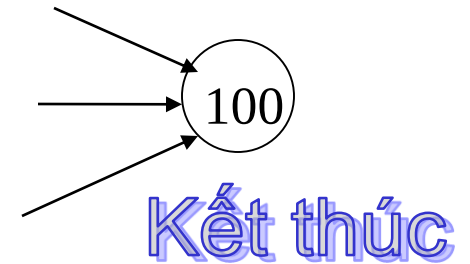
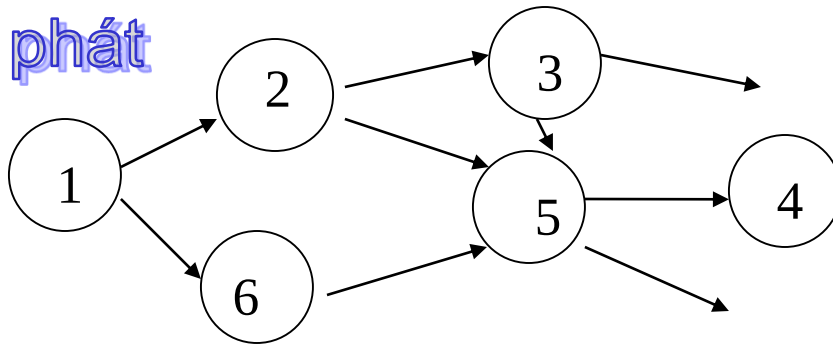


Không gian trạng thái (tiếp)

- Không gian trạng thái
 - Tập hợp các trạng thái có thể nhận biết
 - Trạng thái thực
 - Trạng thái trừu tượng
 - Tập hợp các phép chuyển giữa các trạng thái trong không gian
 - Phép chuyển thực
 - Phép chuyển trừu tượng
 - Giá thành thực hiện một phép chuyển

Ví dụ

Xuất phát



Kết thúc

Phép chuyển
trạng thái

$1 \dashrightarrow 2$

$2 \dashrightarrow 5$

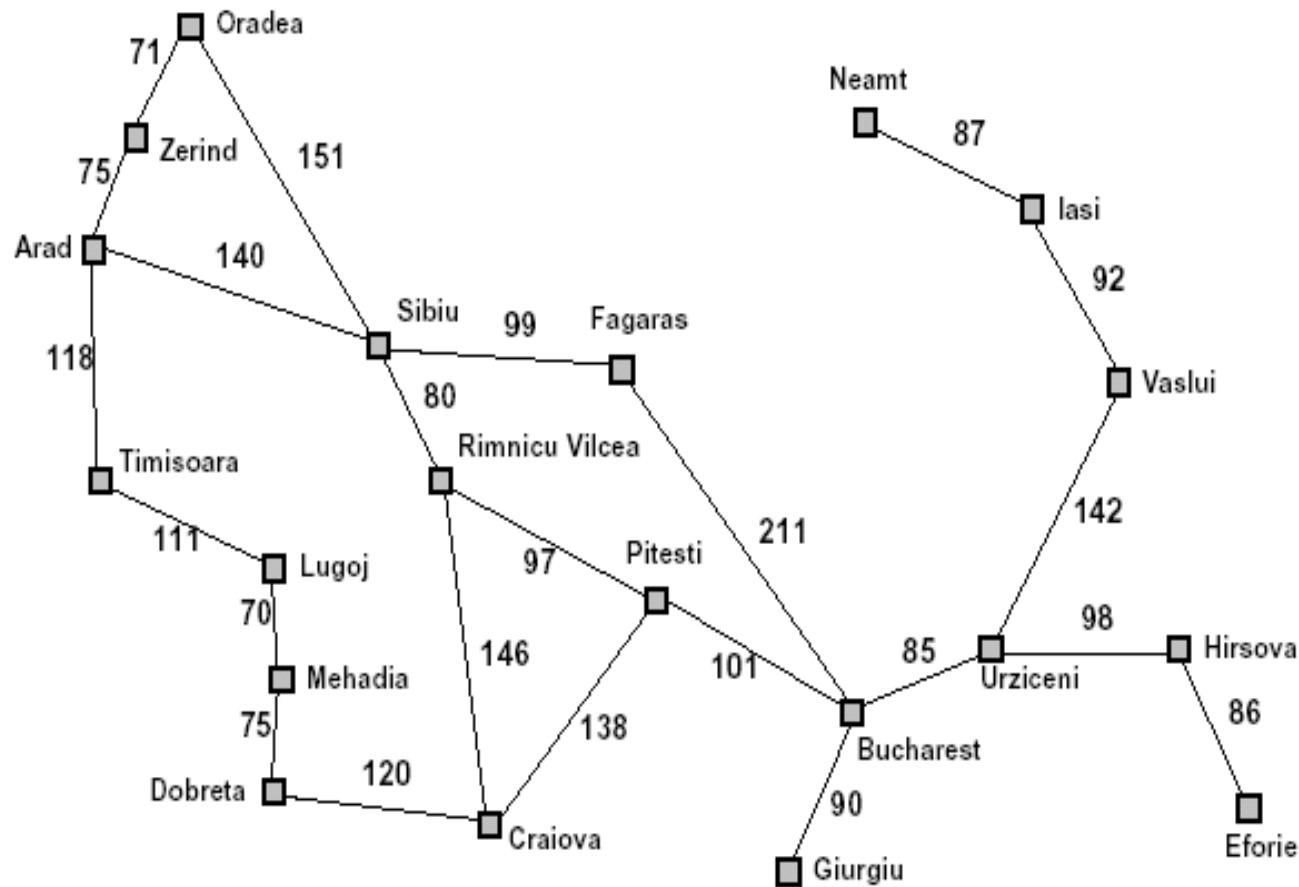
$1 \dashrightarrow 6$

$3 \dashrightarrow 5$

$2 \dashrightarrow 3$

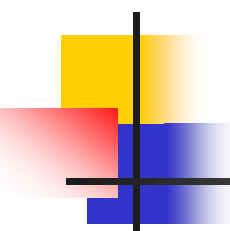
$5 \dashrightarrow 4$

Tìm đường đến Bucarest



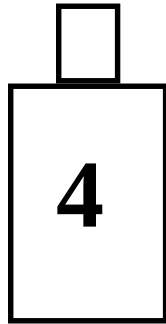
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

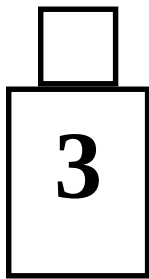


Bài toán rót nước

Cho hai bình nước 3 lít và 4 lít, một máy bơm, một bể nước, hãy tìm giải pháp rót được 2 lít nước vào bình 4 lít



Bình 1



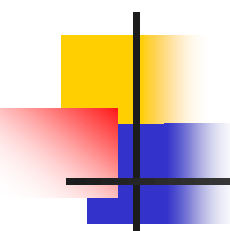
Bình 2



Bơm



Bể nước



Bài toán rót nước (tiếp)

- Không gian trạng thái
 - Trạng thái (x,y) : dung tích nước trong bình 4 lít và 3 lít
 - Phép chuyển trạng thái :
 - Đổ nước trong một bình ra
 - Bơm nước vào một bình
 - Đổ nước từ bình nọ sang bình kia
- Trạng thái xuất phát $(0,0)$
- Trạng thái kết thúc $(2,y)$

Bài toán rót nước (tiếp)

1. $(x, y \mid x < 4) \Rightarrow (4, y)$ Bơm nước vào bình 4

2. $(x, y \mid y < 3) \Rightarrow (x, 3)$ Bơm nước vào bình 3

3. $(x, y \mid x > 0) \Rightarrow (0, y)$ Đổ nước bình 4

4. $(x, y \mid y > 0) \Rightarrow (x, 0)$ Đổ nước bình 3

5. $(x, y \mid x+y \geq 4 \text{ and } y > 0) \Rightarrow (4, y - (4 - x))$

Đổ từ bình 3 sang bình 4 cho tới khi bình 4 đầy

6. $(x, y \mid x+y \geq 3 \text{ and } x > 0) \Rightarrow (x - (3 - y), 3)$

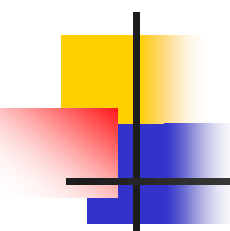
Đổ từ bình 4 sang bình 3 cho tới khi bình 3 đầy

7. $(x, y \mid x+y \leq 4 \text{ and } y > 0) \Rightarrow (x+y, 0)$

Đổ toàn bộ nước từ bình 3 sang bình 4

8. $(x, y \mid x+y \leq 3 \text{ and } x > 0) \Rightarrow (0, x+y)$

Đổ toàn bộ nước từ bình 4 sang bình 3



Giải mã

Gán giá trị cho các chữ cái nhằm thoả mã phép toán

$$\begin{array}{r} \text{F O R T Y} \\ + \quad \text{T E N} \\ + \quad \text{T E N} \\ \hline \text{S I X T Y} \end{array}$$

$$\begin{array}{r} 2 \ 9 \ 7 \ 8 \ 6 \\ + \quad 8 \ 5 \ 0 \\ + \quad 8 \ 5 \ 0 \\ \hline 3 \ 1 \ 4 \ 8 \ 6 \end{array}$$

Giải pháp

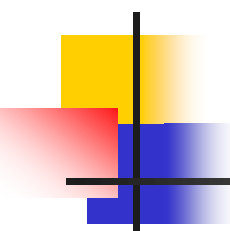
F=2, O=9

R=7, T=8

Y=6, E=5

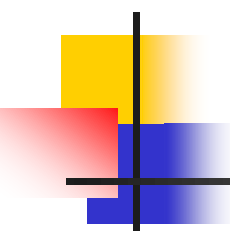
N=0, I=1

X=4



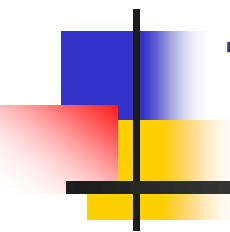
Giải mã (tiếp)

- Không gian trạng thái
 - Trạng thái : Vector gồm các chữ cái và chữ số
 - Phép chuyển : Gán một chữ cái cho một chữ số chưa xuất hiện
- Trạng thái bắt đầu (F,O,R,T,Y,E,N,S,I,X)
- Trạng thái kết thúc : véc tơ số thoả mãn biểu thức
 - VD: (2,9,7,8,6,5,0,1,4)

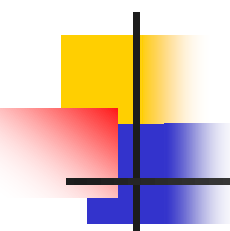


Đồ thị không gian trạng thái

- Nút : trạng thái
- Cung có hướng : phép chuyển trạng thái
- Trọng số cung : giá thành của phép chuyển
- Đặc tính của đồ thị :
 - Cây, đồ thị có hướng không có chu trình, đồ thị có chu trình
 - Số lượng nút
 - Độ phân nhánh : số cung ra lớn nhất
 - Độ sâu : độ sâu tối đa kể từ nút xuất phát

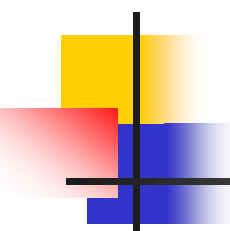


Tìm kiếm trên không gian trạng thái



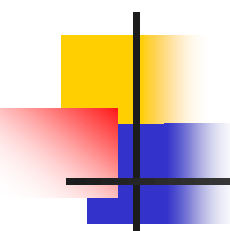
Giải pháp

- Giải pháp cho vấn đề được xác định bằng chuỗi các phép chuyển từ trạng thái xuất phát đến một trạng thái đích
- Tìm giải pháp thực chất là phương pháp tìm kiếm một đường đi từ trạng thái xuất phát đến một trạng thái kết thúc



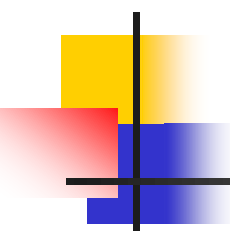
Các giải thuật tìm kiếm

- Tìm kiếm không sử dụng tri thức bổ sung
 - Tìm kiếm rộng
 - Tìm kiếm sâu
 - Tìm kiếm sâu dẫn
 - Tìm kiếm hai chiều
- Tìm kiếm sử dụng tri thức bổ sung
 - Tìm kiếm cực tiểu giá thành
 - Tìm kiếm A^*
 - Tìm kiếm IDA*



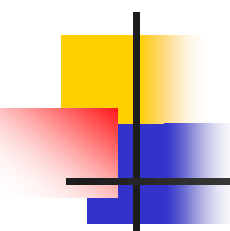
Giải thuật tìm kiếm tổng quát

```
function General-Search(problem) returns solution
    nodes := Make-Queue(Make-Node(Initial-State(problem)))
loop do
    if nodes is empty then return failure
    node := Remove-Front (nodes)
    if Goal-Test[problem] applied to State(node) succeeds
        then return node
    new-nodes := Expand (node, Operators[problem]))
    nodes := Insert-In-Queue(new-nodes)
end
```



Các vấn đề đặt ra

- Xử lý chu trình?
- Khi không gian trạng thái vô hạn?
- Kỹ thuật lựa chọn nút kế tiếp để mở?
- Hướng mở
- Hiệu quả về giá thành?

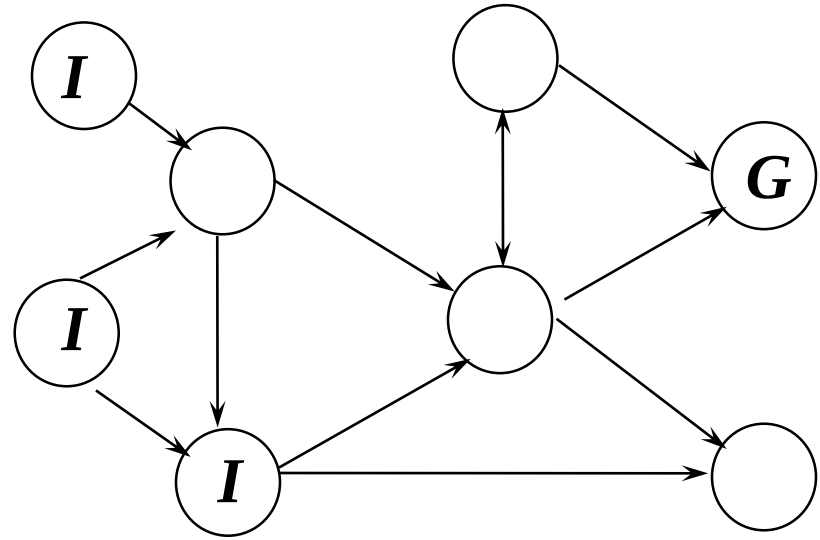
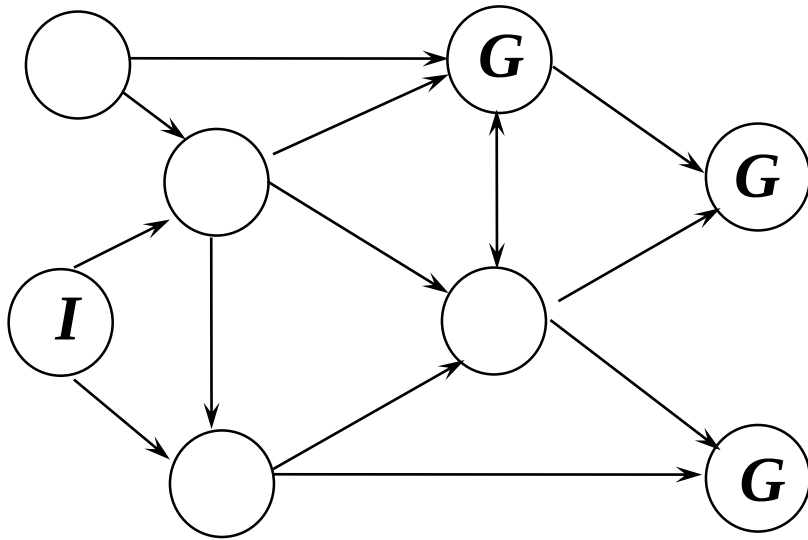


Đánh giá hiệu quả của giải thuật

- Giải thuật có đảm bảo tìm thấy giải pháp (nếu tồn tại) hay không?
- Độ phức tạp tính toán của giải thuật?
- Không gian lưu trữ ?
- Giá thành kết quả?

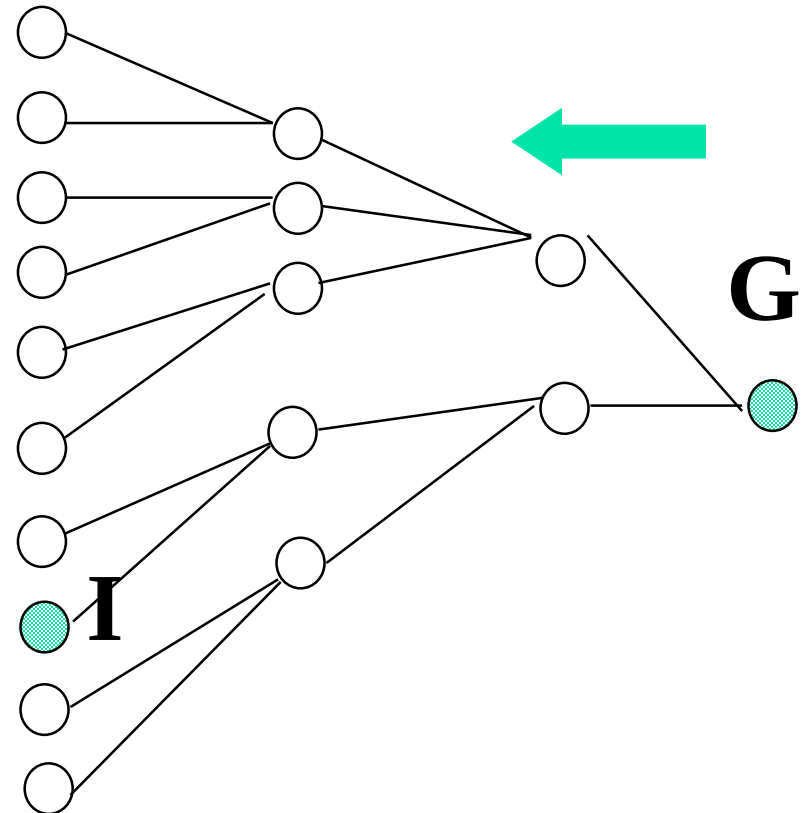
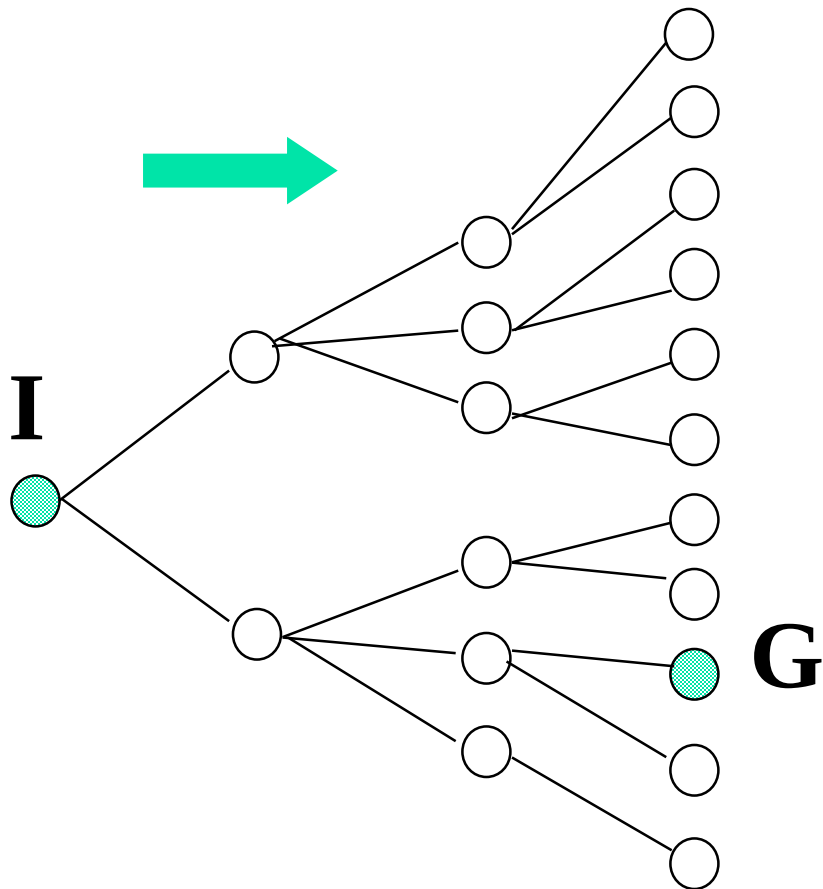
Các yếu tố ảnh hưởng đến hiệu quả

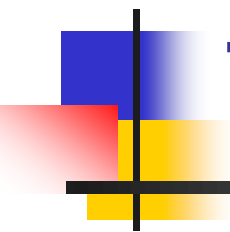
1. Nhiều trạng thái xuất phát hoặc kết thúc



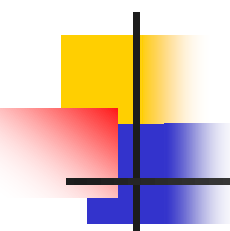
Các yếu tố ảnh hưởng đến hiệu quả (tiếp)

2. Độ phân nhánh





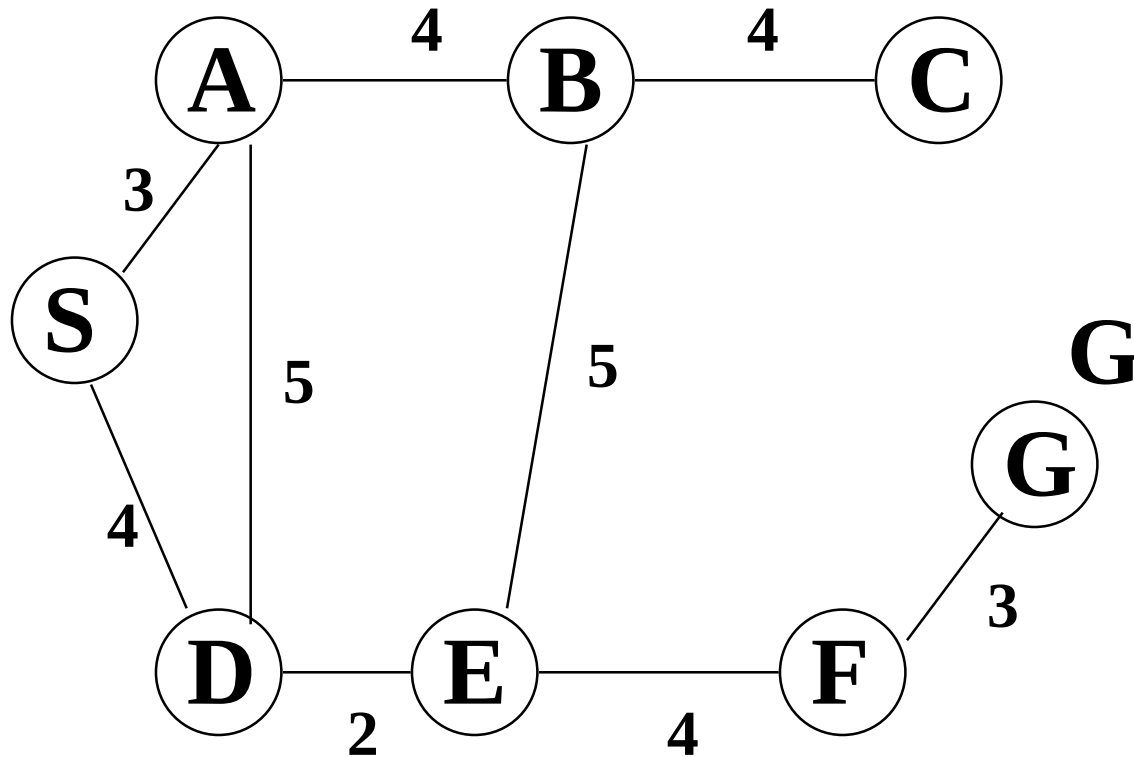
Tìm kiếm không sử dụng tri thức bổ sung



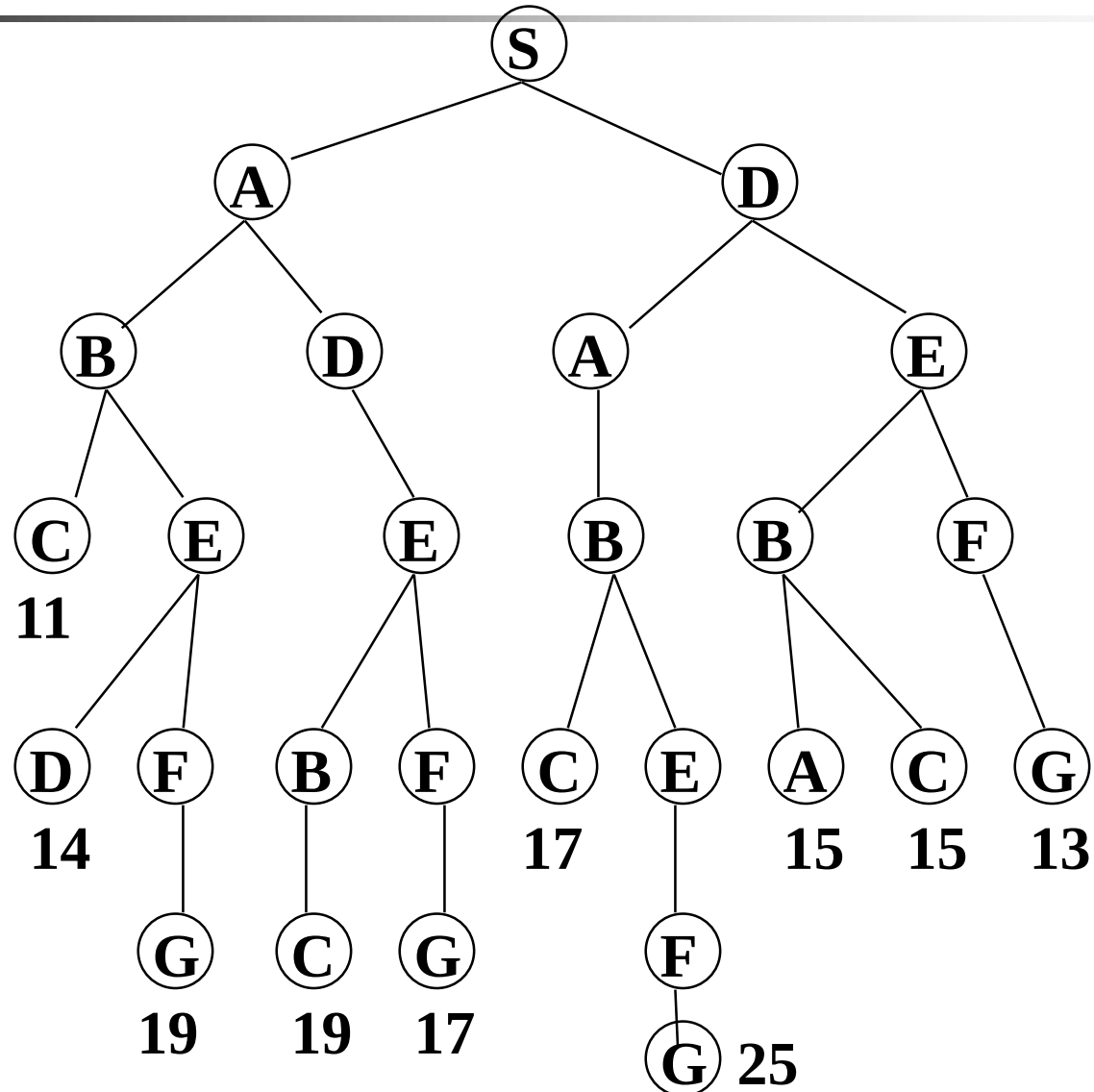
Các phương pháp

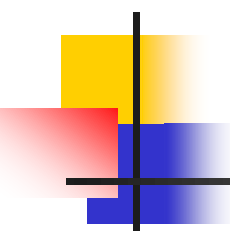
- Tìm kiếm rộng
- Tìm kiếm sâu
- Tìm kiếm sâu dần
- Tìm kiếm hai chiều

Đồ thị không gian trạng thái



Cây tìm kiếm

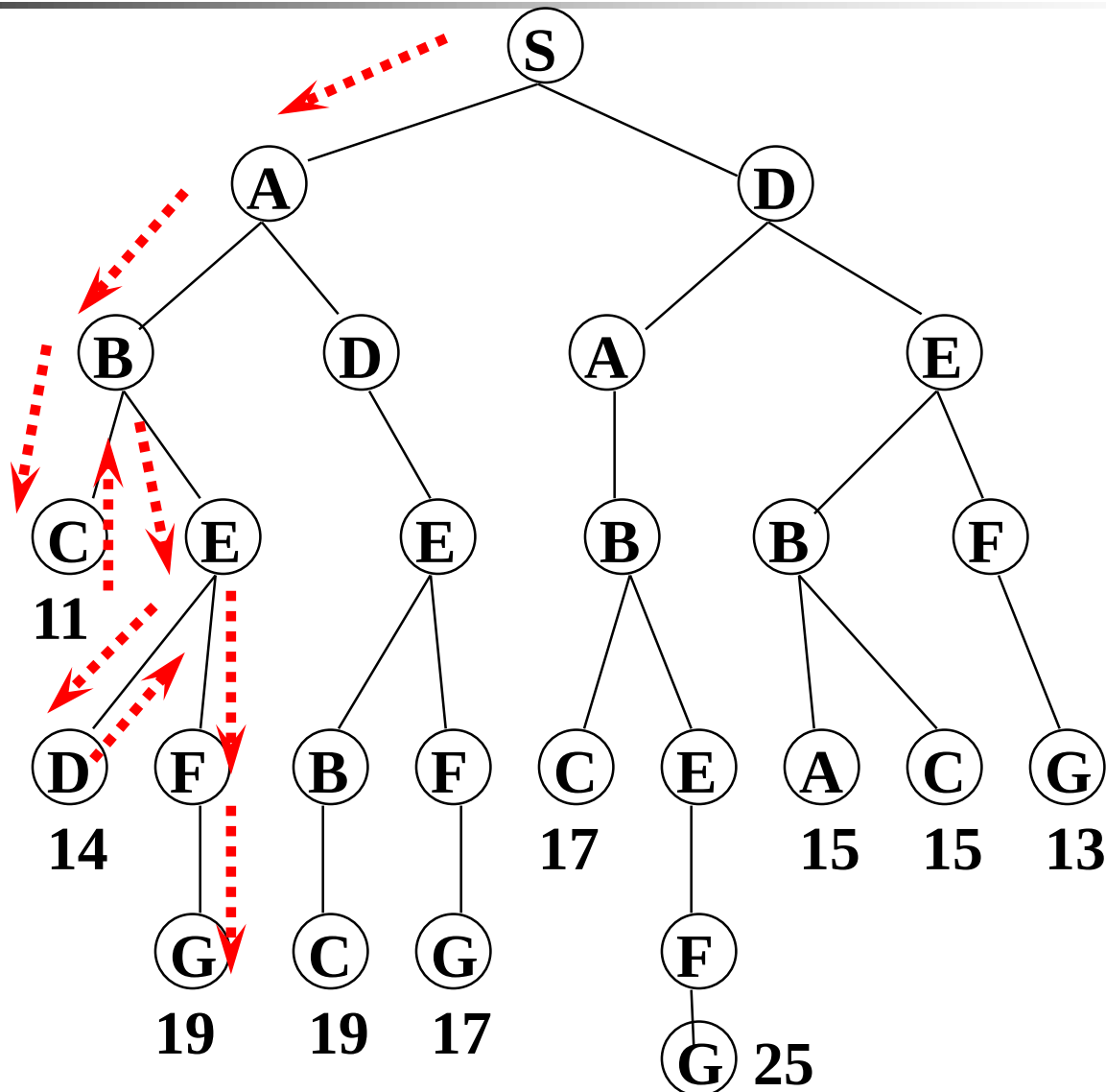


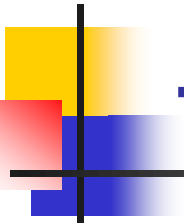


Tìm kiếm sâu

Tiến trên cây tìm kiếm xa nhất có thể, quay lui khi không còn đường

```
function Depth-First-Search(problem) returns solution
  nodes := Make-Queue(Make-Node(Initial-State(problem)))
loop do
  if nodes is empty then return failure
  node := Remove-Front (nodes)
  if Goal-Test[problem] applied to State(node) succeeds
    then return node
  new-nodes := Expand (node, Operators[problem]))
  nodes := Insert-At-Front-of-Queue(new-nodes)
end
```



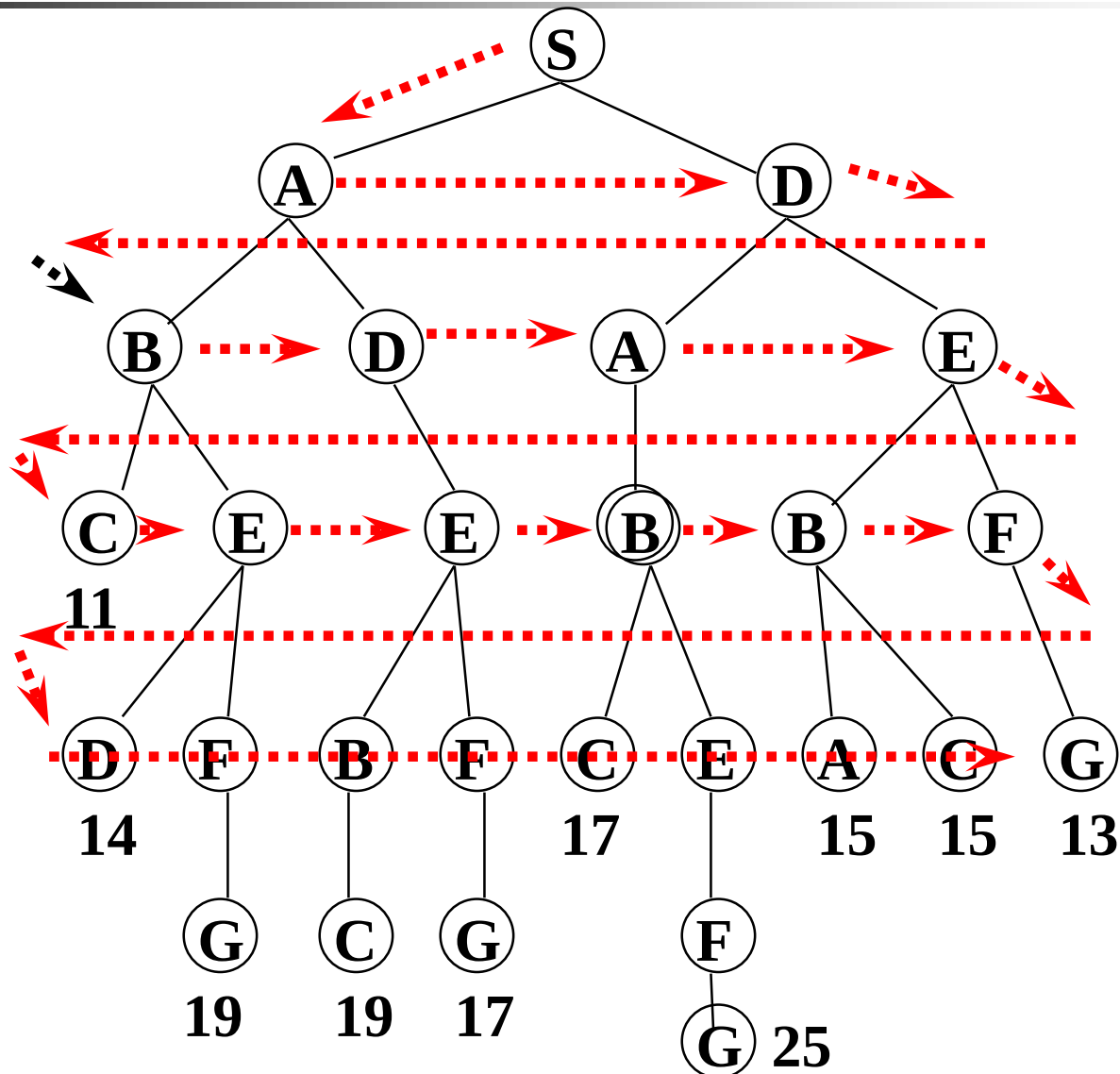


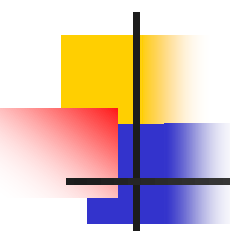
Tìm kiếm rộng

Mở rộng tìm kiếm từng lớp một, tiến hành thử hết lớp thứ n rồi mới tiếp tục mở rộng sang lớp $n + 1$

```
function Breath-First-Search(problem) returns solution
  nodes := Make-Queue(Make-Node(Initial-State(problem)))
loop do
  if nodes is empty then return failure
  node := Remove-Front (nodes)
  if Goal-Test[problem] applied to State(node) succeeds
    then return node
  new-nodes := Expand (node, Operators[problem]))
  nodes := Insert-At-End-of-Queue(new-nodes)
end
```

Tìm kiếm rộng (tiếp)

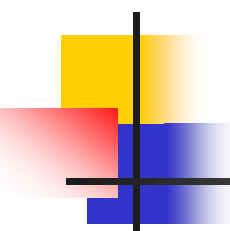




Tìm kiếm sâu dần

- Giống như tìm kiếm sâu nhưng chỉ trong phạm vi độ sâu đã xác định trước.
- Độ sâu của mỗi trạng thái được lưu lại, khi chọn trạng thái tiếp theo để mở, chỉ chọn những trạng thái có độ sâu nhỏ hơn hoặc bằng độ sâu đang chọn.
- Khi tất cả trạng thái ở độ sâu đang chọn, tăng mức độ sâu.
- Kết hợp tìm kiếm rộng và tìm kiếm sâu bằng cách cải thiện giải thuật **Insert-Queue**.

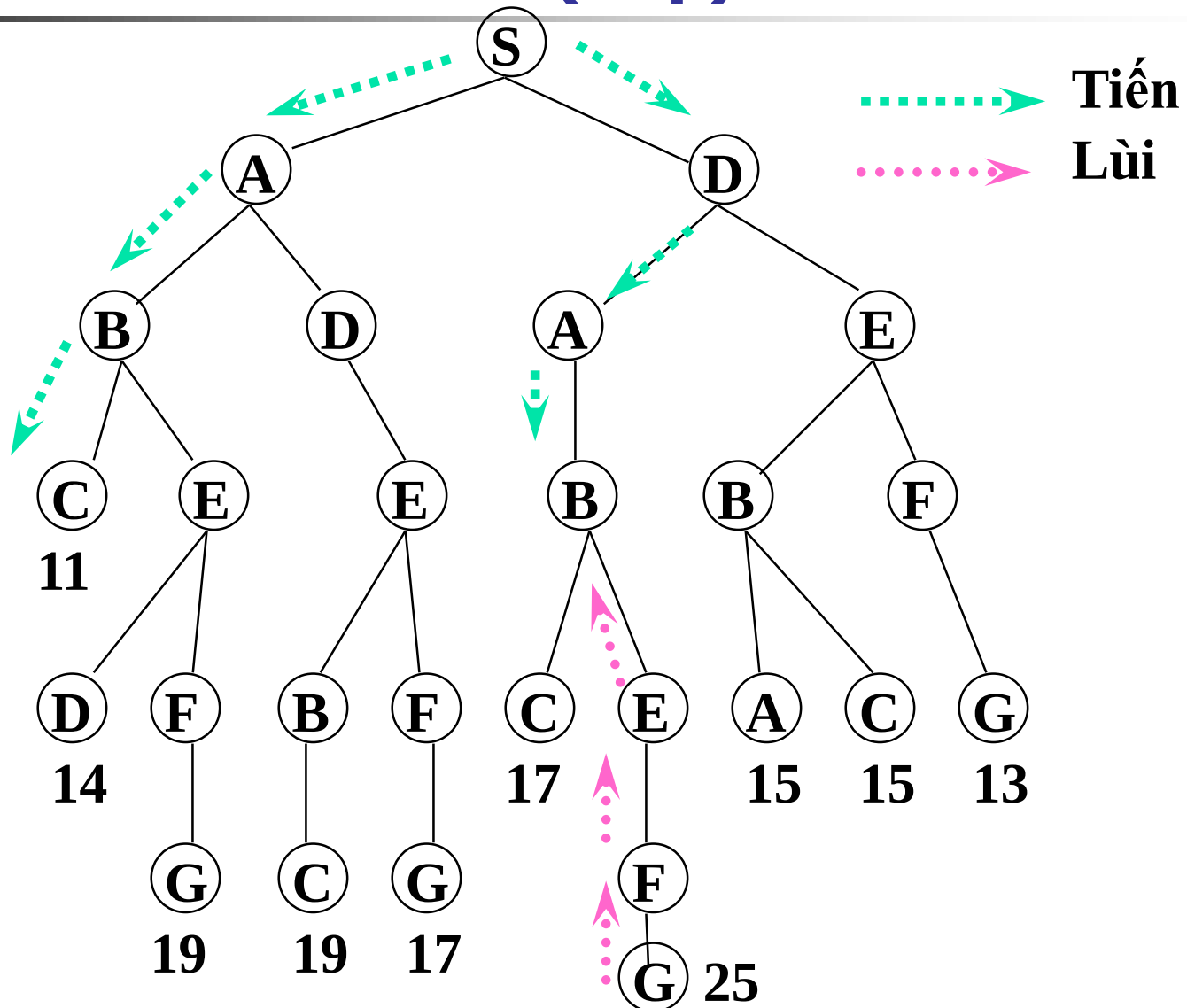


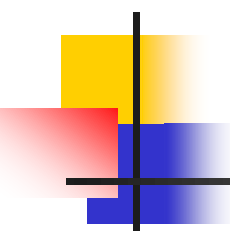


Tìm kiếm hai chiều

- Tìm kiếm đồng thời theo cả hai hướng
- Giải thuật dừng khi chúng gặp nhau tại một trạng thái nào đó
- Chỉ làm việc với những bài toán biểu diễn một trạng thái xuất phát và một trạng thái đích

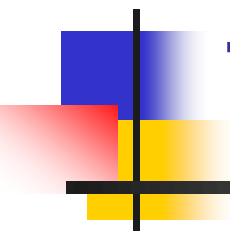
Tìm kiếm hai chiều (tiếp)



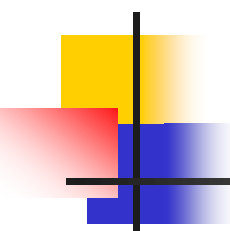


Xử lý chu trình

- Chu trình có thể gây nên những thao tác tìm kiếm lặp lại
- Xử lý
 - Không mở những trạng thái vừa mở
 - Không mở những trạng thái nằm trên đường đi
 - Không mở những trạng thái đã được mở



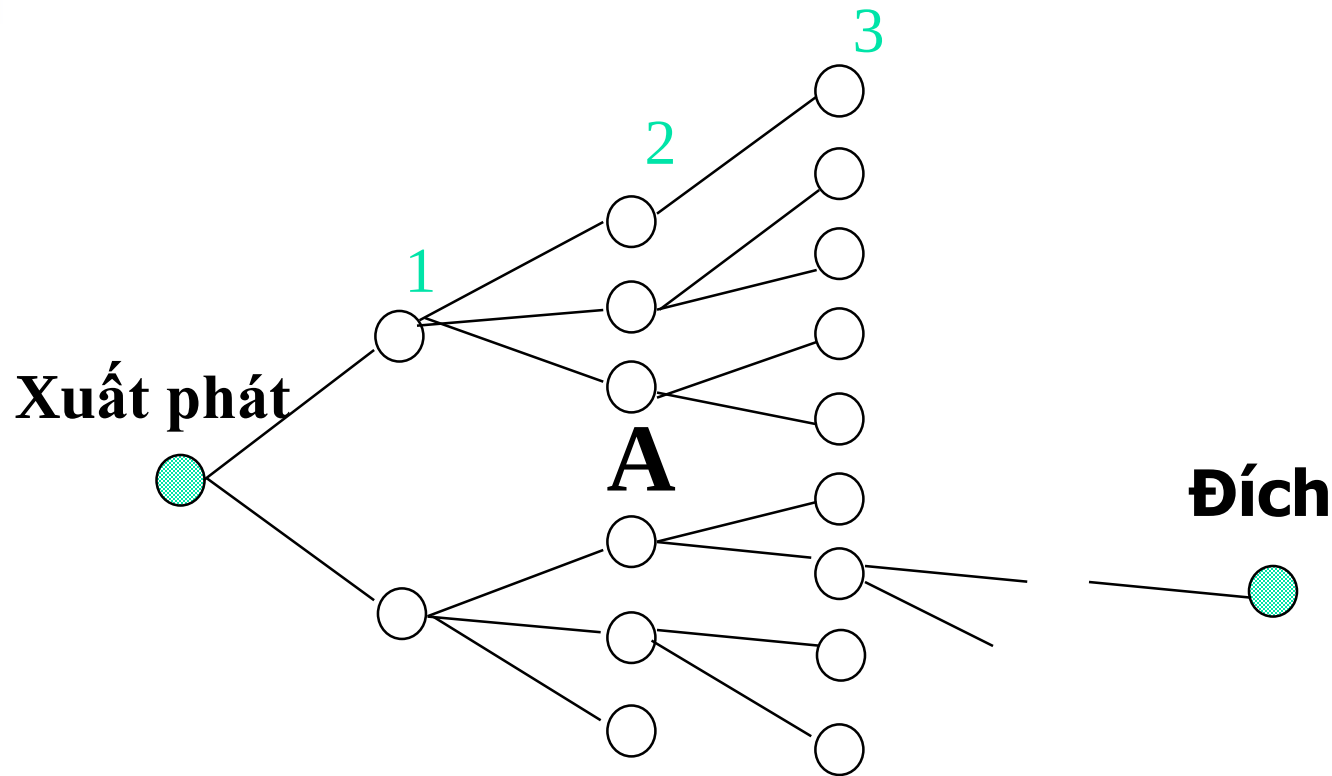
Tìm kiếm sử dụng tri thức bổ sung



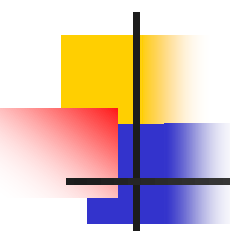
Hạn chế của phương pháp tìm kiếm không sử dụng tri thức bổ sung

- Không khai thác được bản chất vấn đề
- Chỉ mở rộng theo những hướng xác định, không phù hợp với những sự kiện xảy ra trên đường đi

Tri thức bổ sung

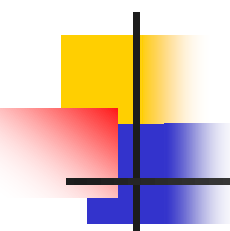


Nếu như biết A rất có triển vọng đến kết quả, hãy tìm cách mở nó.



Tri thức bổ sung (tiếp)

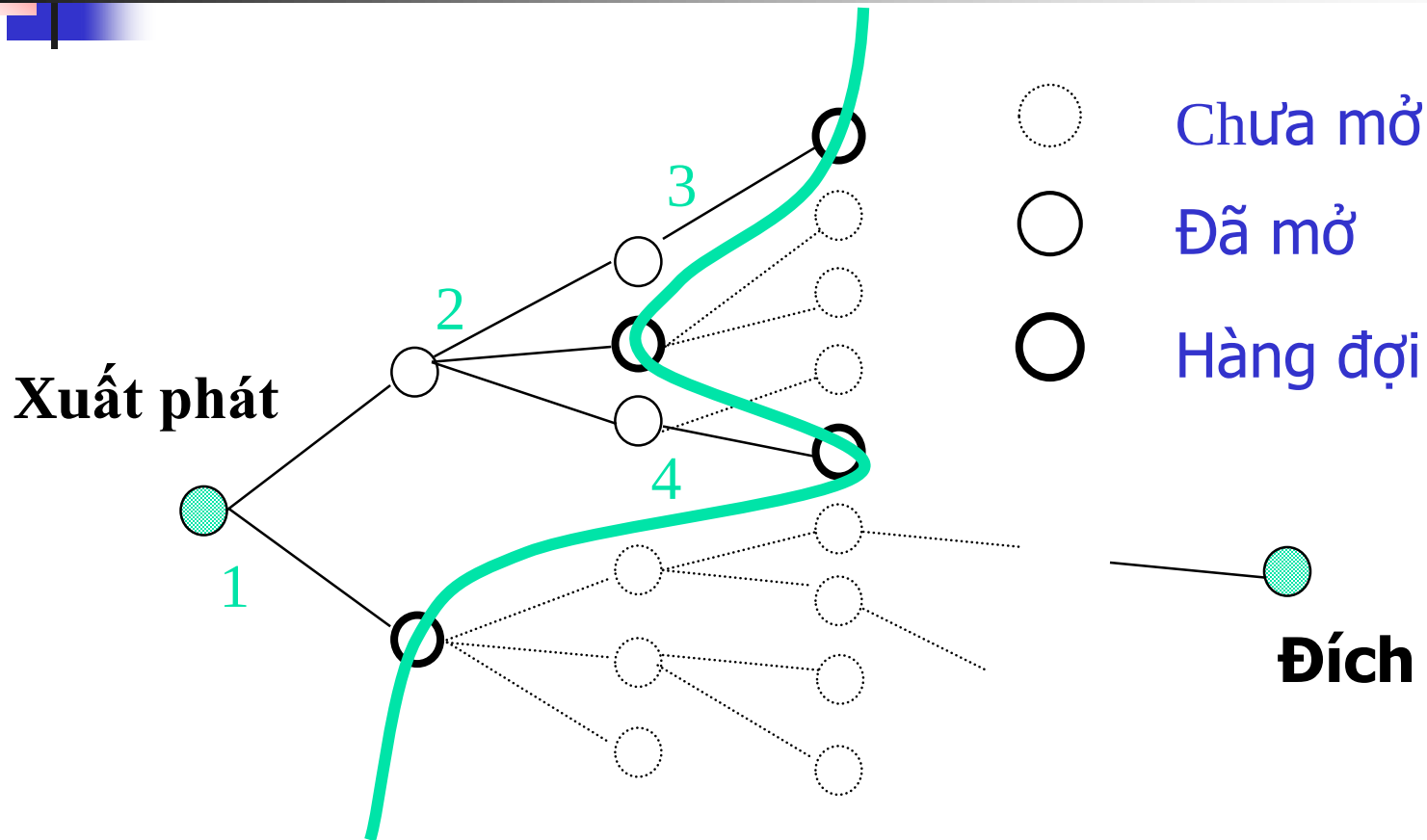
- Heuristic : các luật cho phép **có thể** tiếp cận đích nhanh hơn
- Heuristic xuất phát từ bản chất của vấn đề
- Heuristic càng chính xác sẽ cho kết quả càng tốt

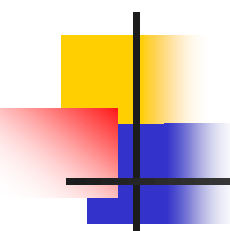


Tìm kiếm cực tiểu

```
function Best-First-Search(problem, Eval-FN)
    returns solution sequence
    nodes := Make-Queue(Make-Node(Initial-State(problem)))
loop do
    if nodes is empty then return failure
    node := Remove-Front(nodes)
    if Goal-Test[problem] applied to State(node) succeeds
        then return node
    new-nodes := Expand(node, Operarors[problem],
                        Eval-FN)))
    nodes := Insert-by-Cost(new-nodes, Eval-FN(new-node))
end
```

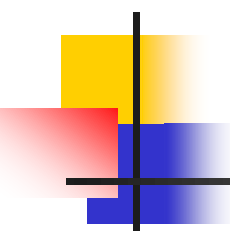

Tìm kiếm cực tiểu (tiếp)





Tìm kiếm cực tiểu (tiếp)

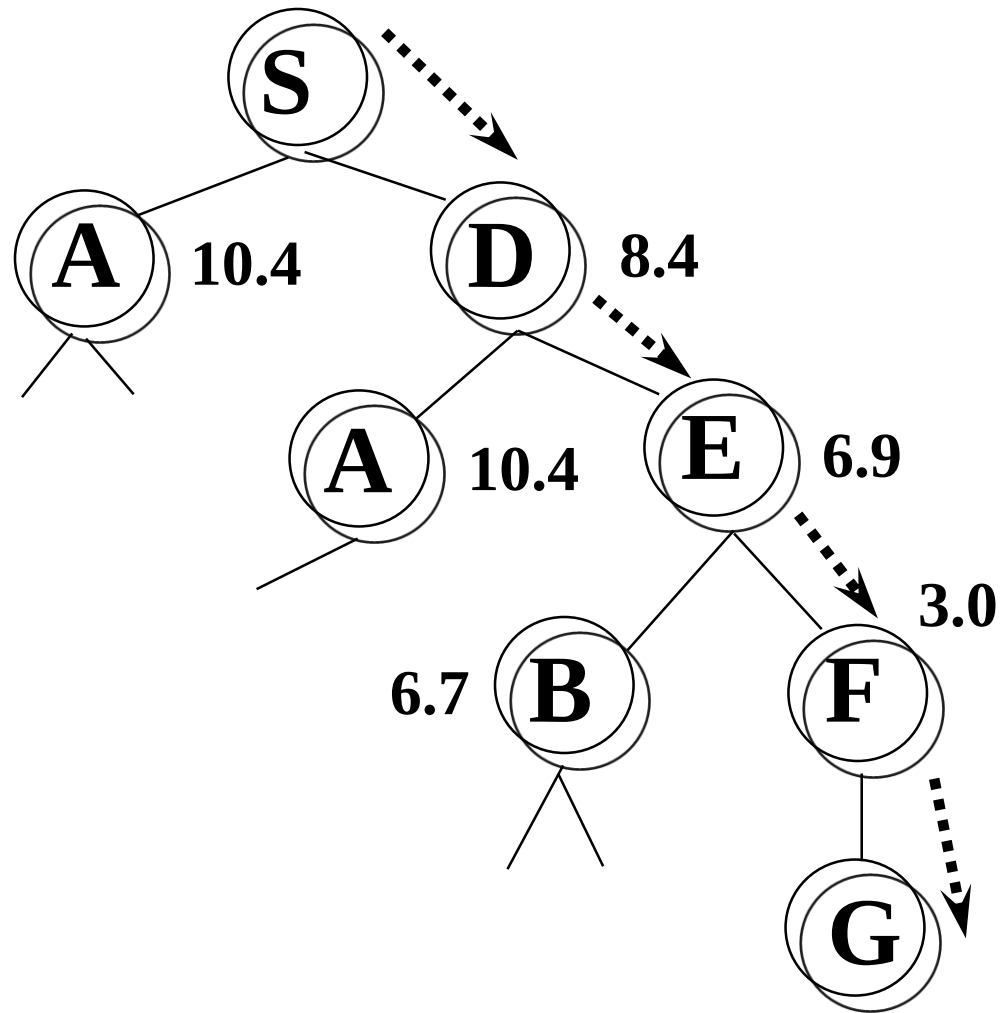
- Hàm heuristic (Eval-FN)
 - $g(n)$: Chi phí từ khi xuất phát đến nút hiện tại n
 - $h(n)$: Chi phí dự đoán từ nút n đến đích
 - $f(n) = h(n) + g(n)$

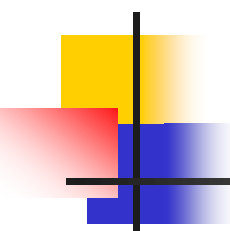


Giải thuật tham lam

- Mở nút có giá trị $h(n)$ nhỏ nhất
- Lựa chọn mang tính chất cục bộ
- Chưa chắc đảm bảo tối ưu
- Có thể chuyển nhanh đến đích

Giải thuật tham lam - $h(n)$

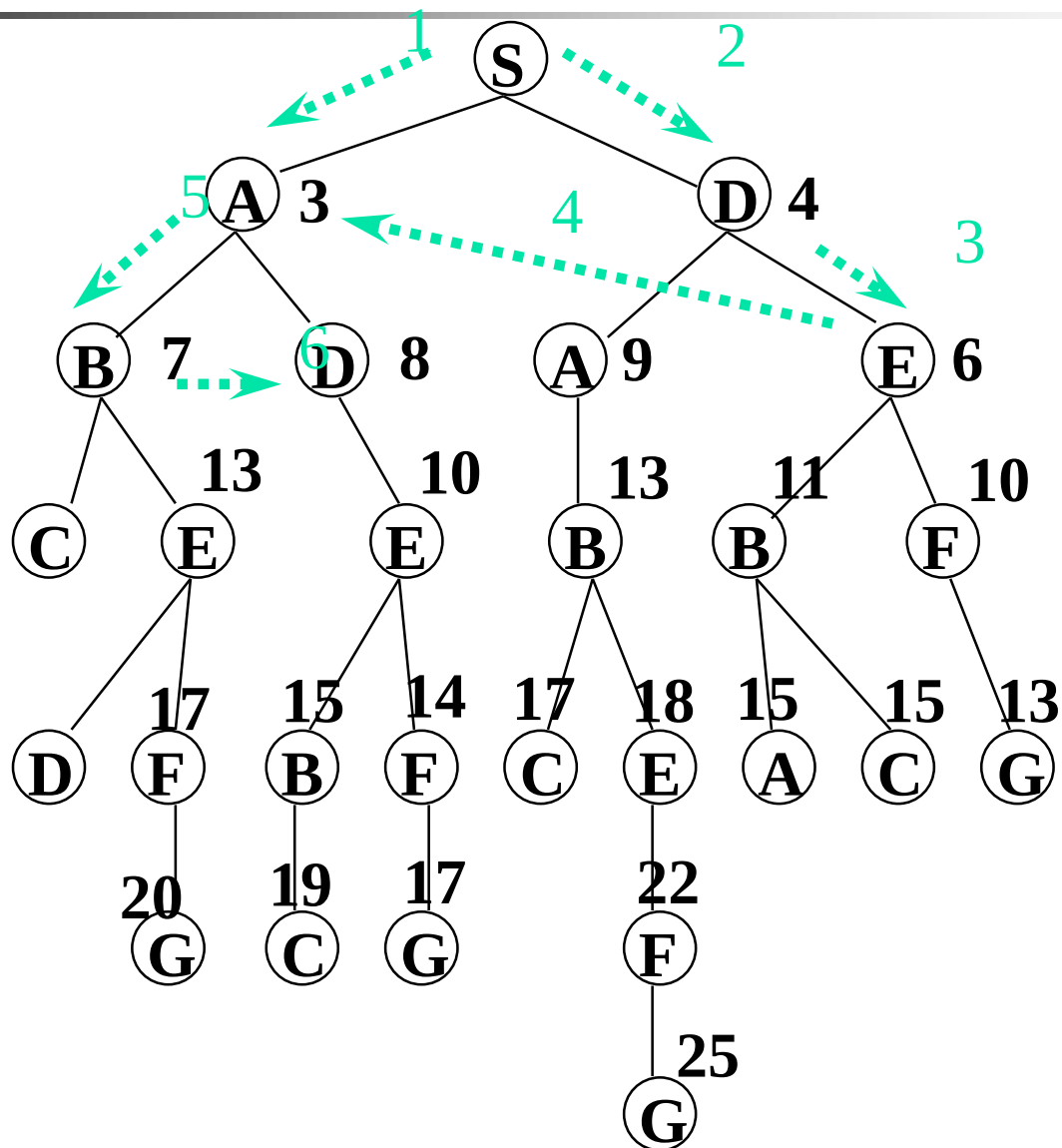


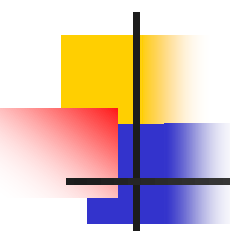


Giải thuật nhánh cận

- Mở nút có giá trị $g(n)$ nhỏ nhất
- Không sử dụng heuristic
- Cho kết quả tối ưu

Giải thuật nhánh cận

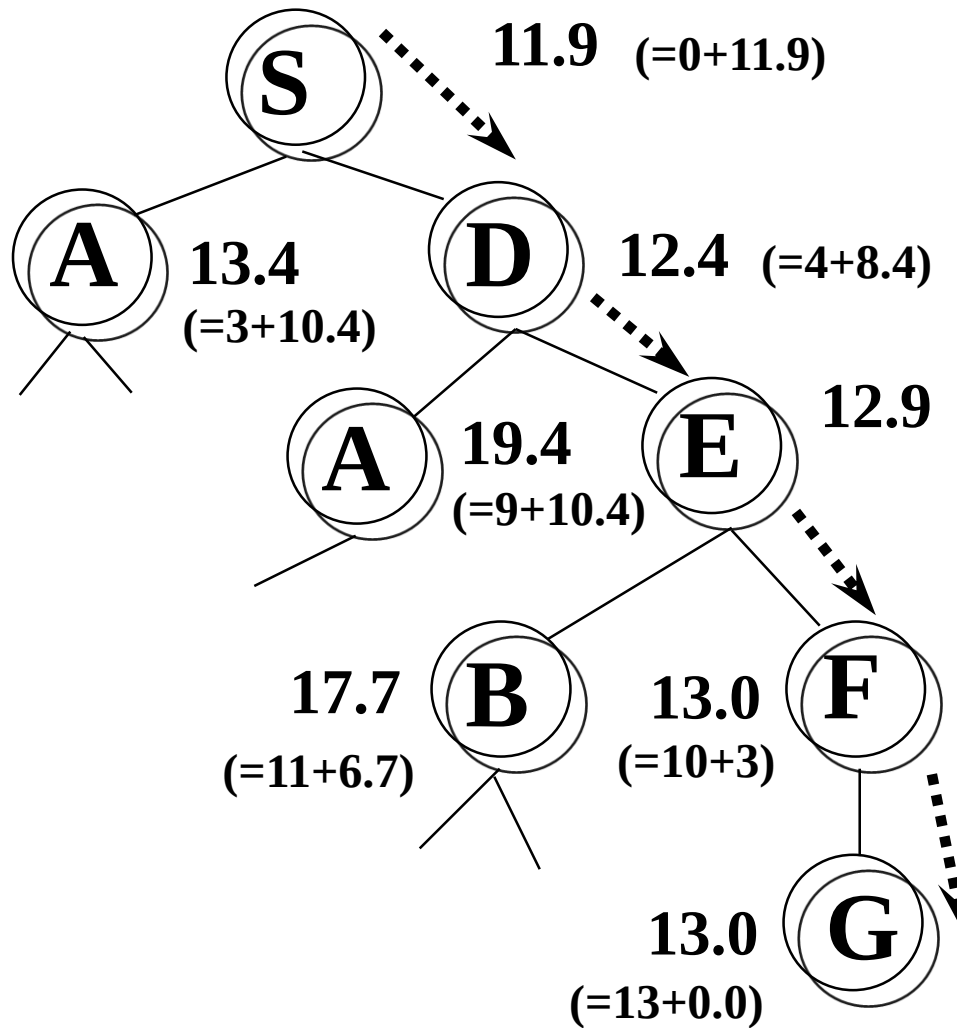




Giải thuật heuristic A*

- Chọn nút để mở là nút có $f(n)$ nhỏ nhất
- Nếu heuristic là chính xác thì ta có giải pháp tốt nhất với chi phí tốt nhất

Giải thuật heuristic A*



Ví dụ về hàm heuristic $h(n)$

- h_1 số ô sai vị trí
- h_2 tổng khoảng cách từ ô hiện thời đến ô đúng vị trí

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

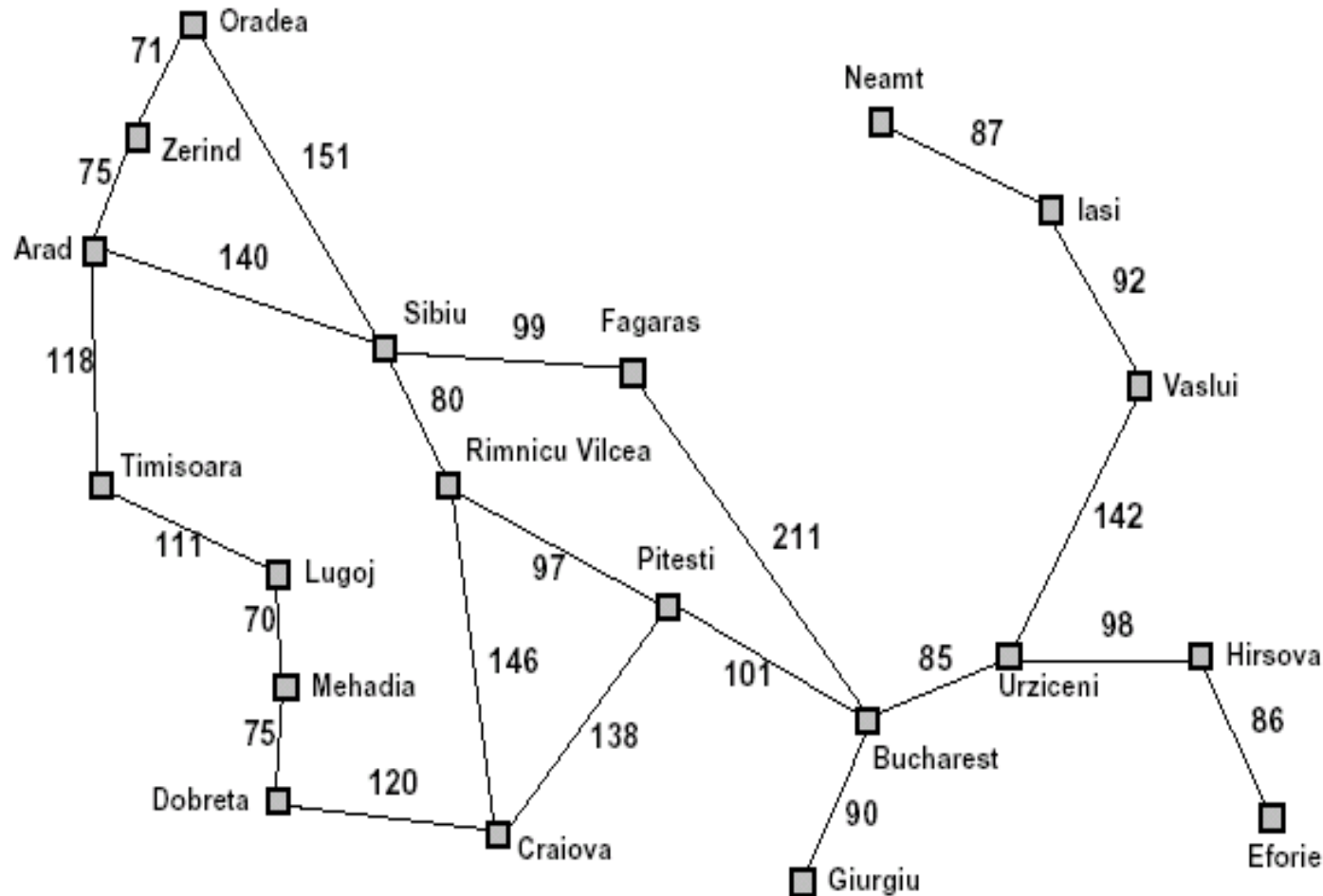
Goal State

$$h_1 = 7$$

$$h_2 = 19$$

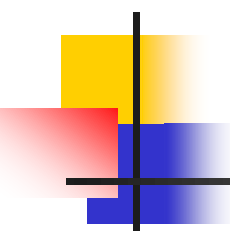
$$(2+3+3+3+4+2+0+2)$$

Ví dụ về hàm heuristic $h(n)$



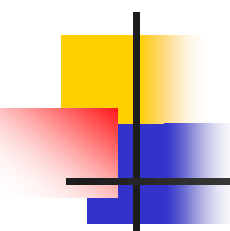
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



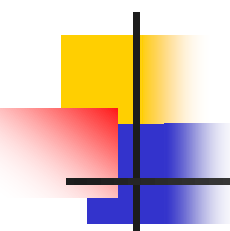
Tìm kiếm A* sâu dần (IDA*)

```
function IDA*(problem) returns solution  
    root := Make-Node(Initial-State[problem])  
    f-limit := f-Cost(root)  
loop do  
    solution, f-limit := DFS-Contour(root, f-limit)  
    if solution is not null, then return solution  
    if f-limit = infinity, then return failure  
end
```

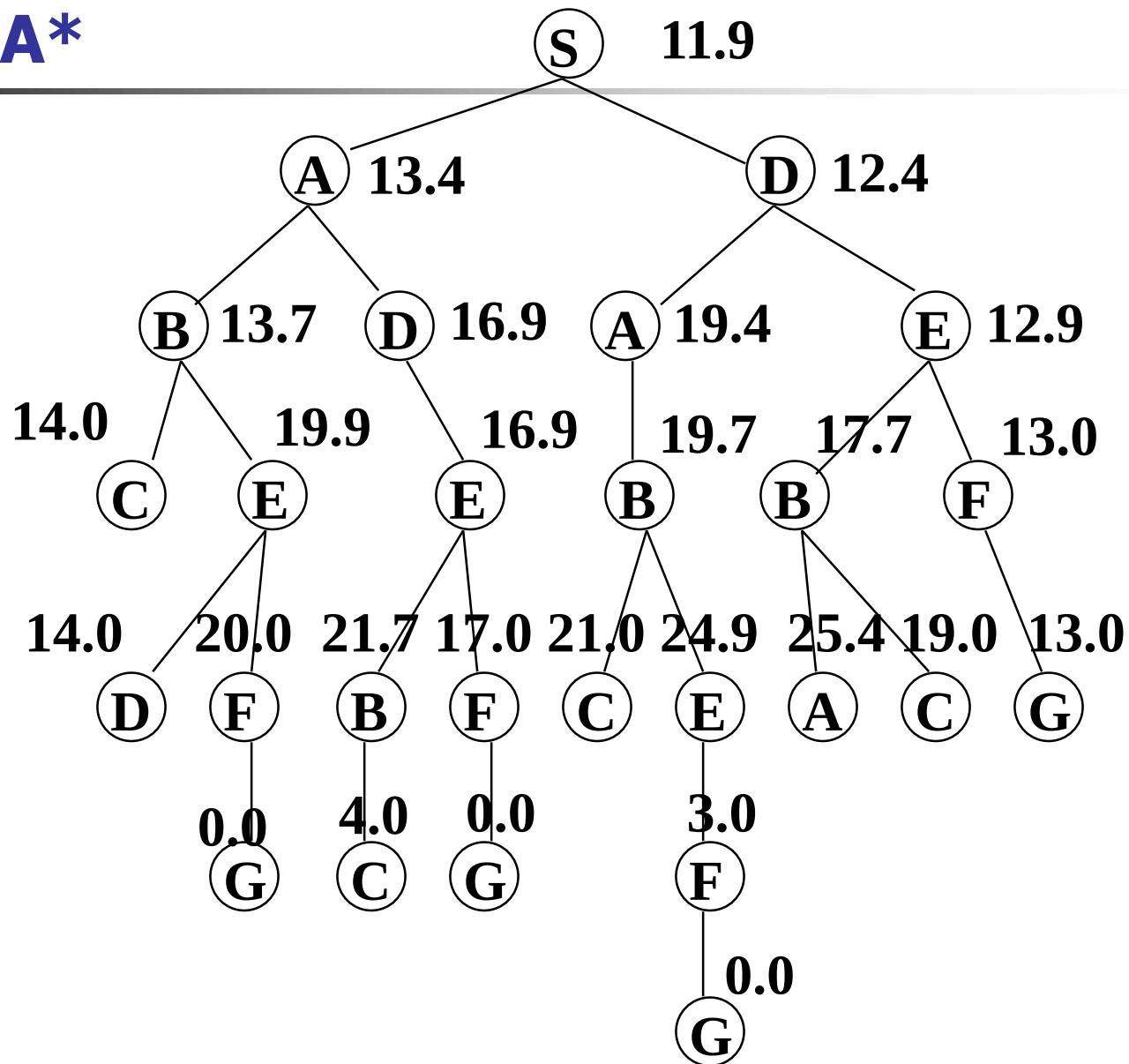


Tìm kiếm A* sâu dần (IDA*)

```
function DFS-Countour(node, f-limit) returns  
    solution sequence and new f-limit  
if f-Cost[node] > f-limit then return (null, f-Cost[node] )  
if Goal-Test[problem](State[node]) then return (node, f-limit)  
for each node s in Successor(node) do  
    solution, new-f := DFS-Contour(s, f-limit)  
    if solution is not null, then return (solution, f-limit)  
    next-f := Min(next-f, new-f); end  
return (null, next-f)
```



IDA*





IDA*

Mức 0:

IDA(S, 11.9)

11.9

Mức 1:

IDA(S, 11.9)

IDA(A, 11.9)

13.4

IDA(D, 11.9)

12.4

Mức 2:

IDA(S, 12.4)

IDA(A, 12.4)

13.4

IDA(D, 12.9)

IDA(A, 12.9)

19.4

IDA(E, 12.9)

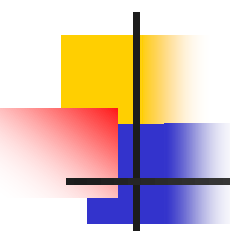
12.9

Mức 3:

IDA(S, 12.9)

IDA(F, 12.9)

13.0

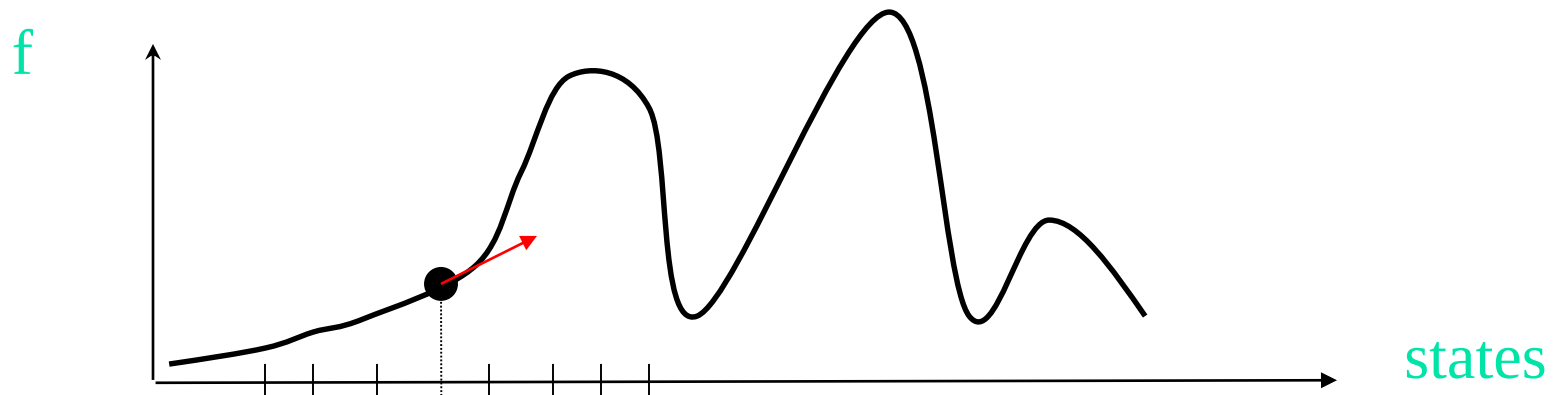


Tìm kiếm bằng cách cải thiện dần dần

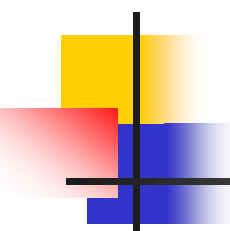
- Sử dụng khi không có sự đánh giá heuristic
- Chỉ có đánh giá duy nhất về trạng thái chuyển tốt hơn
- Xuất phát từ một trạng thái hợp lệ sau đó cải thiện dần dần
 - Bài toán xác định cực trị

Giải thuật leo đồi

- Áp dụng luật cho phép tăng giá trị f lớn nhất có thể
- Dịch chuyển theo hướng véc tơ gradient

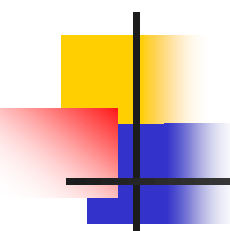


```
while f-value(state) > f-value(best-next(state))  
    state := next-best(state)
```

Giải thuật leo đồi

- Còn gọi là giải thuật giảm gradient
- Cho cực trị cục bộ
- Có thể cải thiện tính cục bộ bằng cách
 - Nhảy ngẫu nhiên
 - Quay lui
 - Sử dụng mô men quán tính



Các giải thuật không tiền định

- Giải thuật di truyền (genetic algorithm)
- Giải thuật phỏng tôì (simulated annealing)